

Agenda

- 1) VS Code → Workspace + Environment
- 2) Pydantic → Python Code

Langgraph → Pydantic

KRIM

- 1) Environments → Projects, → Conda

Env 1 ←
Project 1

Python: 3.10

Libraries

Langchain

Pandas

Numpy

Env 2 ←
Project 2

Python: 3.12

—

—

—

—

Env 3 ←
Project 3

Python 3.13

—

—

—

—

Agenda

- 1) Langchain
- 2) Langsmith
- 3) Open AI API, GROK,
- 4) Coding

1) Google → Gemini ✓ ←

2) Meta → Llama ✓

3) AWS → Cloud

4) Anthropic → Claude ✓

5) Open AI → GPT ✓

6) MICROSOFT → Cloud ✓

7) GROK → ✓ ←

Building Applications

F415 AWS funding Anthropic

Microsoft funds OpenAI

Assistant → LLM → Pydantic →

Product → {product name: " "}
product details: " "

Tentative price: USD → Int or float

Assistant

ChatPrompt template

Create a simple assistant that uses any LLM and should be pydantic, when we ask about any product it should give you two information product Name, product details tentative price in USD (integer). use chat Prompt Template.

Setup

Groq cloud = Deployments

Hugging Face = Github for AI

Langchain = Develop Gen AI applications

Langgraph = Built on top of Langchain
- Develop Agentic AI Apps

Langsmith = Monitoring

API keys;

Always save API keys inside .env file.

Q. How to get API Key?

Groq cloud → API Key (provides opensource models)

Langsmith → Settings → API Key (to use Langsmith)
cloud.

Open AI → Platform → API Key

Imp. Langchain project → "project_Name"

LIBRARIES REQUIRED

```
requirements.txt
1 pydantic
2 python-dotenv
3 langchain
4 langchain-openai
5 langchain-groq
6 langchain-core
7 langchain-community
```

Access .env variables

Models

Prompts etc.

dotenv = from dotenv import load_dotenv
load_dotenv() → load all the variables

```
## Langsmith Tracking And Tracing
os.environ["GROQ_API_KEY"]=os.getenv("GROQ_API_KEY")
os.environ["LANGCHAIN_API_KEY"]=os.getenv("LANGCHAIN_API_KEY")
os.environ["OPENAI_API_KEY"]=os.getenv("OPENAI_API_KEY")
os.environ["LANGCHAIN_PROJECT"]=os.getenv("LANGCHAIN_PROJECT")
os.environ["LANGCHAIN_TRACING_V2"]="true" #tells LangChain to turn on its tracing system (v2).

# Tracing means keeping a step-by-step record of how a program runs.>
```

Turn on Tracing

Import ChatOpenAI

```
from langchain_openai import ChatOpenAI

llm=ChatOpenAI(model="o1-mini")

result=llm.invoke("What is Agentic AI in short")
print(result)
```

✓ 2.8s

content='Agentic AI refers to artificial intelligence systems

Import ChatGroq

```
from langchain_groq import ChatGroq

# Use a known Groq production/free model
model = ChatGroq(model="llama-3.1-8b-instant")

response = model.invoke("what is open ai")
print(response.content)
```

✓ 0.9s

OpenAI is an American artificial intelligence (AI)

Prompt Engineering; is the process of designing and refining prompts (instructions) to the LLM to get accurate and useful outputs.

Key Aspects of Prompt Engineering:

1. **Clarity** – Writing unambiguous prompts so the model knows exactly what you need.
 - Example: Instead of "Tell me about sales," → "Summarize the top 3 strategies e-commerce companies use to increase sales in 2024."
2. **Context** – Giving the model background or role-based instructions.
 - Example: "You are a data analyst. Explain SQL joins to a beginner using simple business examples."
3. **Constraints** – Setting boundaries like format, tone, or length.
 - Example: "Write in bullet points, under 100 words, in a professional tone."
4. **Iteration** – Refining the prompt step by step until the output matches expectations.

Advanced Prompting Techniques

Here's the table version of the **Advanced Prompt Engineering Techniques**:

Technique	Description	Example
Chain-of-thought prompting	Guides the model to explain reasoning step by step.	"Solve this math problem step by step: 23×17 ."
Few-shot prompting	Provides a few examples in the prompt so the model learns the pattern.	"Translate English to French: Cat → Chat, Dog → Chien, Bird → Oiseau. Now translate 'Horse'."
Zero-shot prompting	Asks directly without giving any examples.	"Translate 'Horse' into French."
Role prompting	Assigns the model a specific role or persona to guide responses.	"You are a job interviewer. Ask me SQL questions one by one."

⚡ Why It Matters

- Better prompts → Better results.
- Saves time by reducing irrelevant or incorrect answers.
- Critical skill for **GenAI**, **agentic AI**, and **LLM-powered applications** (e.g., chatbots, copilots, analytics assistants).

Two parts; system & user

```
### Prompt Engineering
from langchain_core.prompts import ChatPromptTemplate

prompt=ChatPromptTemplate.from_messages(
    [
        ("system","You are an expert AI Engineer. Provide me answer based on the question"),
        ("user","{input}")
    ]
)
prompt
```

Let's Build first

Project



Complete Project Life Cycle

```
# Initiate Pydantic Class for outputparser
class Product_details(BaseModel):
    product_name: str
    product_configurations: str
    tentative_price_usd: int
```

} Pydantic for Type Hinting
(output parser)

```
from langchain_core.output_parsers import JsonOutputParser
```

```
# Initialize Output Parser with Pydantic schema
```

```
output_parser = JsonOutputParser(pydantic_object=Product_details)
```

→

parser
+
Pydantic

```
# Initiate Model
```

```
model=ChatOpenAI(temperature=0.7)
```

```
# Initiate Chat Prompt Template
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a helpful assistant that can answer questions about products
    and must provide information strictly in the format specified."""),
    ("user", "Give the details of {input}.\n{format_instructions}") #format instructions is very imp here
])
```

Query

↳ to enforce parser instructions

```
# Chain
```

```
chain = prompt | model | output_parser
```

```
# Invoke with dict input (matches {query} and {input} in your prompt)
```

```
response = chain.invoke({
    "query": "Give details",
    "input": "iphone 13",
    "format_instructions": output_parser.get_format_instructions() → v. imp.
})
```

```
print(response)
```

```
{'product_name': 'iPhone 13', 'product_configurations': 'Various configurations available with different storage'}
```


framework to
build applications

LANGCHAIN

Glues everything Together

LangChain is an **open-source framework** designed to build applications powered by **Large Language Models (LLMs)** like GPT-4, Claude, Gemini, etc.

It helps developers connect LLMs with:

- **External data sources** (databases, PDFs, APIs, websites)
- **Tools** (search engines, calculators, custom APIs)
- **Memory** (to keep context across conversations)
- **Chains/Workflows** (multi-step reasoning pipelines)

◆ Key Components

1. **LLM Wrappers** → APIs for GPT, Claude, Gemini, etc.
2. **Prompt Templates** → standardize and reuse prompts.
3. **Chains** → sequence of steps (e.g., summarize → search → generate answer).
4. **Agents** → LLMs that decide which tools to use dynamically.
5. **Memory** → store conversation history or state.
6. **Retrievers** → connect external knowledge (e.g., vector databases like Pinecone, FAISS).

◆ Why use LangChain?

Out of the box, an LLM can:

- Take a prompt → return an answer.

But real-world apps need more:

- Look up **company data** from **SQL/ElasticSearch**
- Search the web in real time
- Use **long documents** with Retrieval-Augmented Generation (RAG)
- Maintain **conversational memory** (chatbots)
- Orchestrate **multi-step tasks**

◆ Example Use Cases

- **Chatbots** that answer from your company's internal docs.
- **AI assistants** that can call APIs, use tools, and retrieve knowledge.
- **Document Q&A** (upload PDF → ask natural language questions).
- **Code assistants** with multi-step reasoning.

✓ In short:

LangChain = framework that **supercharges** LLMs by giving them memory, tools, and access to external knowledge → making them useful ↓ production-grade applications.



Imp

"LLMs are trained on a fixed snapshot of internet-scale data, but they don't know about a company's private or latest information. To make them relevant for enterprise use cases, we integrate them with company-specific data using techniques like RAG. Frameworks like LangChain make this integration easier by connecting LLMs with external knowledge bases (databases, PDFs, APIs, vector stores), enabling context-aware, accurate, and business-specific responses."

Langchain vs RAG

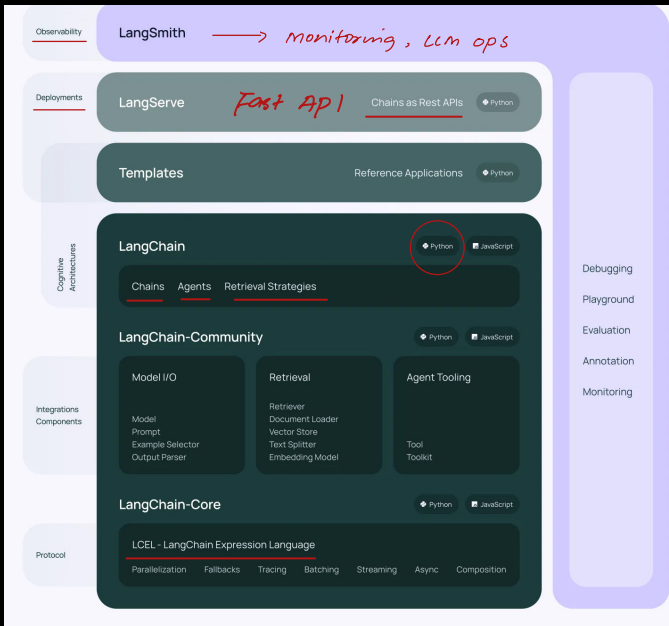
Aspect	RAG (Retrieval-Augmented Generation)	LangChain
What it is	A technique/approach to improve LLMs by retrieving external data and feeding it into the prompt.	A framework/library that helps you build apps with LLMs (implements RAG + other workflows).
Purpose	Make LLMs relevant to company/domain-specific knowledge.	Simplify development of GenAI apps by providing ready-made modules.
Core Idea	1. Embed documents → store in vector DB → retrieve relevant chunks → add to prompt → generate answer.	Provides tools (chains, memory, agents, retrievers, loaders) to implement RAG, chatbots, agents, workflows.
Dependency	Conceptual — can be coded from scratch without any library.	Implementation — often uses RAG as one of its key techniques.
Example	"Chat with company policies" → Retrieve policy docs → LLM answers using retrieved info.	Build the same chatbot in 10 lines of code using LangChain RetrievalQA module.
Analogy	RAG = Recipe (method of cooking).	LangChain = Kitchen tools that make cooking faster.

🧠 LLMs & Their Companies

Company	Model Family / Names
OpenAI	GPT-3, GPT-3.5, GPT-4, GPT-4o, GPT-5
Anthropic	Claude 1, Claude 2, Claude 3, Claude 3.5
Google DeepMind	Gemini 1, Gemini 1.5, Gemini 2
Meta (Facebook)	LLaMA 1, LLaMA 2, LLaMA 3
Mistral AI	Mistral 7B, Mixtral (MoE), Mistral Next
Cohere	Command R, Command R+, Command R+ (2024/25)
xAI (Elon Musk)	Grok (Grok-1, Grok-1.5, Grok-2)
Anthropic + Partners	Claude models integrated into Amazon (Bedrock)
Amazon	Titan family (Bedrock platform also hosts Anthropic, Cohere, etc.)
Alibaba	Qwen (Tongyi Qianwen)

7102562104

Langchain Ecosystem



🔑 LangChain Components in a Nutshell

Part	What it does	Example
LLMs	Connects to language models (OpenAI, Claude, Gemini, etc.)	Use GPT-4 for text generation
Prompt Templates	Pre-defined prompt structures for consistency	"Summarize this text in 3 bullet points: {text}"
Chains	Sequence of steps combining prompts, models, tools	Summarize → Translate → Save to DB
Agents	LLM decides which tool/action to use dynamically	Chatbot that picks between search or calculator
Memory	Stores past interactions to maintain context	Chatbot remembers previous messages
Retrievers	Finds relevant info from external sources	Search in PDFs, SQL, or vector DBs
VectorStores	Stores embeddings for similarity search (FAISS, Pinecone, Chroma)	"Chat with PDF" feature
Tools	External APIs or functions that LLM can call	Weather API, SQL query, calculator

In Short:

Langsmith = Monitoring, LLM op

Langserve = APIs - Fast API

Langchain = chain, Agents

Langchain Core = Langchain Exp.